

UD05.02: Profundización en la POO

Contenido

1. Atributos y métodos.....	2
1.1 Atributos de instancia y de clase	2
1.2 Métodos de instancia, clase y estáticos	3
2. Encapsulamiento y acceso a datos	5
2.1 Atributos públicos, protegidos y privados	5
2.2 Getters y setters	9
3. Decoradores para Control de Atributos	10
3.1 @property.....	10
3.2 Setter con @atributo.setter	10
3.3 Eliminación con @atributo.setter	11
4. Herencia	12
4.1 Herencia simple	12
4.2 Uso de super().....	12
4.3 Herencia múltiple.....	15
4.4. Funciones issubclass() y isinstance()	15
4.5 Delegación	16
5. Polimorfismo.....	17
6. Abstracción.....	19
7. Gestión de recursos y destrucción de objetos.....	21
7.1 Método __del__	21
7.2 Gestores de contexto y método with	21
8. Métodos especiales en Python	23
8.1 Representación de objetos: __str__ y __repr__	23
8.2 Comparación de objetos: __eq__ y operadores relacionales	25
8.3 Operaciones con objetos: __add__ y __sub__	27
8.4 Otros métodos especiales	28
Qué es el mangling.....	29
¿Qué es el MRO (Method Resolution Order)?	31

1. Atributos y métodos

1.1 Atributos de instancia y de clase

Tipos de atributos:

- Los **atributos de clase** o estáticos son compartidos por todas las instancias de la clase. Y son aquellos definidos dentro de la clase, pero no dentro de un método. Estas variables son compartidas por todos los objetos de una clase. Los atributos de clase son muy útiles para:
 - Contar instancias creadas.
 - **Mantener configuraciones comunes.**
 - Compartir información general entre todos los objetos.
- Los **atributos de instancia** pertenecen a cada objeto y se definen normalmente dentro del constructor (`__init__`).

Ejemplo:

```
class Libro:
```

```
    # Atributo de clase: contador de libros creados
```

```
    contador_libros = 0
```

```
    def __init__(self, titulo, autor):
```

```
        # Atributos de instancia
```

```
        self.titulo = titulo
```

```
        self.autor = autor
```

```
    # Cada vez que se crea un libro, aumenta el contador
```

```
    Libro.contador_libros += 1
```

```
# Se crean tres objetos
```

```
libro1 = Libro("Cien años de soledad", "Gabriel García Márquez")
```

```
libro2 = Libro("El Principito", "Antoine de Saint-Exupéry")
```

```
libro3 = Libro("1984", "George Orwell")
```

```
# El atributo de clase refleja cuántos libros se han creado
print("Libros creados:", Libro.contador_libros)
```

1.2 Métodos de instancia, clase y estáticos

Tipos de métodos:

- **Instancia (self):** Accede a atributos y métodos del objeto.
- **Clase (@classmethod):** Usa cls como primer parámetro y accede a atributos de clase.
 - cls hace referencia a **la propia clase**, no a una instancia específica.
- **Estáticos (@staticmethod)** o de clase: No reciben ni self ni cls. Son aquellos que no necesitan acceso a ningún atributo de ningún objeto en concreto de la clase.

Ejemplo:

```
class Ejemplo:
```

```
    contador_clase = 0 # Este es un atributo de clase
```

```
    def __init__(self, nombre):
```

```
        # Este es un método de instancia (constructor)
```

```
        # 'self' hace referencia a la instancia del objeto.
```

```
        # 'self.nombre' es un atributo de instancia.
```

```
        self.nombre = nombre
```

```
        Ejemplo.contador_clase += 1 # Accede y modifica el atributo de clase
```

```
    def saludar(self):
```

```
        # Este es un método de instancia
```

```
        # Accede a 'self.nombre', que es un atributo de instancia.
```

```
        return f"Hola, soy una instancia llamada {self.nombre}"
```

```
    @classmethod
```

```
    def total_instancias(cls):
```

```
        # Este es un método de clase
```

```
        # 'cls' hace referencia a la clase misma (Ejemplo en este caso).
```

```
# Accede a 'cls.contador_clase', que es un atributo de clase.  
return cls.contador_clase
```

```
@staticmethod
```

```
def mensaje_estatico():
```

```
    # Este es un método estático
```

```
    # No recibe 'self' ni 'cls'. No puede acceder a atributos de instancia ni de clase.
```

```
    return "Soy un mensaje de un método estático."
```

```
# Uso de los métodos:
```

```
# Métodos de Instancia
```

```
instancia1 = Ejemplo("PrimeraInstancia")
```

```
print(instancia1.saludar())
```

```
instancia2 = Ejemplo("SegundaInstancia")
```

```
print(instancia2.saludar())
```

```
# Método de Clase
```

```
# Se llama directamente desde la clase o desde una instancia
```

```
print(f"Número total de instancias creadas: {Ejemplo.total_instancias()}")
```

```
print(f"Número total de instancias (desde instancia): {instancia1.total_instancias()}")
```

```
# Método Estático
```

```
# Se llama directamente desde la clase
```

```
print(Ejemplo.mensaje_estatico())
```

2. Encapsulamiento y acceso a datos

2.1 Atributos públicos, protegidos y privados

En Python, la visibilidad de los atributos (y métodos) de una clase **se maneja por convención** o a través de un mecanismo de "mangling" (ver explicación del concepto al final del documento) de nombres. No existen modificadores de acceso estrictos como en otros lenguajes (como public, protected, private en Java o C++). En cambio, se utilizan prefijos para indicar la intención del programador.

Tipos de acceso:

- **Público**: accesible desde cualquier parte.
- **Protegido** (prefijo `_`): por convención, no debe accederse desde fuera.
- **Privado** (doble guion bajo `__`): no accesible directamente desde fuera.

Ejemplo1:

```
class Cuenta:
```

```
    def __init__(self, saldo):
```

```
        self._saldo = saldo    # Protegido
```

```
        self.__clave = "secreta" # Privado
```

```
cuenta = Cuenta(1000)
```

```
print(cuenta._saldo)    # Válido, aunque no recomendado
```

```
# print(cuenta.__clave) # Error: atributo privado
```

Ejemplo 2:

class Cuenta:

```
def __init__(self, saldo_inicial, numero_cuenta):
```

```
    # Atributo Público: accesible desde cualquier parte.
```

```
    # Se usa para datos que forman parte de la interfaz pública del objeto.
```

```
    self.numero_cuenta = numero_cuenta
```

```
    # Atributo Protegido (por convención, prefijo _):
```

```
    # Indica que el atributo está destinado a uso interno de la clase
```

```
    # y sus subclases. No se recomienda acceder a él directamente desde fuera.
```

```
    self._saldo = saldo_inicial
```

```
    # Atributo Privado (doble guion bajo __):
```

```
    # Python "manglea" el nombre de este atributo para que no sea accesible
```

```
    # directamente desde fuera de la clase. Es para información interna.
```

```
    self.__clave_secreta = "supersecreta123"
```

```
def mostrar_saldo(self):
```

```
    # Método público que accede a un atributo protegido
```

```
    print(f"Número de cuenta: {self.numero_cuenta}")
```

```
    print(f"Saldo actual: {self._saldo} €")
```

```
def _depositar(self, cantidad):
```

```
    # Método protegido: se espera que sea usado internamente o por subclases.
```

```
    if cantidad > 0:
```

```
        self._saldo += cantidad
```

```
        print(f"Depósito de {cantidad} € realizado. Nuevo saldo: {self._saldo} €")
```

```
    else:
```

```
        print("La cantidad a depositar debe ser positiva.")
```

```
def __verificar_clave(self, clave):
```

```
    # Método privado: solo accesible desde dentro de la propia clase.
```

```
return clave == self.__clave_secreta
```

```
def retirar(self, cantidad, clave):
```

```
    # Método público que utiliza un método privado
```

```
    if self.__verificar_clave(clave):
```

```
        if cantidad > 0 and self._saldo >= cantidad:
```

```
            self._saldo -= cantidad
```

```
            print(f"Retiro de {cantidad} € realizado. Nuevo saldo: {self._saldo} €")
```

```
        else:
```

```
            print("Cantidad inválida o saldo insuficiente.")
```

```
    else:
```

```
        print("Clave incorrecta. Acceso denegado.")
```

```
# --- Ejemplos de Acceso ---
```

```
print("--- Creando una cuenta ---")
```

```
mi_cuenta = Cuenta(saldo_inicial=1000, numero_cuenta="ES123456789")
```

```
# Acceso a atributo público: Totalmente permitido y esperado.
```

```
print(f"Número de cuenta (público): {mi_cuenta.numero_cuenta}")
```

```
# Acceso a atributo protegido: POSIBLE, pero NO RECOMENDADO.
```

```
# Python no lo impide, pero la convención indica que no deberías hacerlo.
```

```
print(f"Saldo (protegido - acceso directo NO recomendado): {mi_cuenta._saldo} €")
```

```
# Acceso a atributo privado: Intento directo falla (genera AttributeError).
```

```
# print(mi_cuenta.__clave_secreta) # Descomentar esto causará un error
```

```
# Acceso a métodos públicos
```

```
mi_cuenta.mostrar_saldo()
```

```
mi_cuenta.retirar(200, "supersecreta123")
```

```
mi_cuenta.retirar(500, "clave_erronea")
```

Acceso a método protegido: POSIBLE, pero NO RECOMENDADO.

mi_cuenta._depositar(300) # Aunque posible, mejor usar un método público si existe uno
mi_cuenta.mostrar_saldo()

Acceso al atributo privado a través del mangling de nombres (NO RECOMENDADO)

Solo con fines ilustrativos, no lo uses en código real.

print(mi_cuenta._Cuenta__clave_secreta)

2.2 Getters y setters

Para implementar la encapsulación y no permitir el acceso directo a los atributos, se ponen los atributos ocultos y se declaran métodos específicos para acceder y modificar esos atributos (mutadores). Estos métodos se denominan getters y setters. Y permiten controlar el acceso a atributos privados.

Ejemplo:

```
class Producto:
```

```
    def __init__(self, precio):  
        self.__precio = precio
```

```
    def get_precio(self):  
        return self.__precio
```

```
    def set_precio(self, valor):  
        if valor >= 0:  
            self.__precio = valor
```

```
p = Producto(50)
```

```
p.set_precio(60)
```

```
print(p.get_precio()) # 60
```

3. Decoradores para Control de Atributos

Python ofrece **decoradores** para transformar métodos en propiedades, lo que permite un control más sofisticado sobre cómo se acceden y modifican los atributos de una clase.

3.1 @property

Convierte un método en un atributo de solo lectura.

Ver ejemplo en el apartado siguiente.

3.2 Setter con @atributo.setter

Permite asignar valor controlado al atributo.

Ejemplo:

```
class Producto:
```

```
    def __init__(self, precio):
```

```
        self._precio = precio
```

```
    @property
```

```
    def precio(self):
```

```
        return self._precio # Indirectamente se está creando un getter, ya que devuelve el valor
```

```
    @precio.setter
```

```
    def precio(self, valor):
```

```
        if valor >= 0:
```

```
            self._precio = valor
```

```
p = Producto(100)
```

```
p.precio = 150 # Parece un acceso directo, pero realmente usa el setter
```

```
print(p.precio) # 150, Parece una lectura directa, pero usa el getter
```

3.3 Eliminación con @atributo.setter

Python permite definir una función *deleter* asociada a un atributo para controlar lo que sucede cuando este es eliminado mediante `del`.

Para ello, se usa el decorador `@atributo.deleter` junto con la definición del método correspondiente.

Ejemplo:

```
class Circulo:
```

```
    def __init__(self, radio):
```

```
        self._radio = radio # Atributo "privado" (convención con guion bajo)
```

```
    @property
```

```
    def radio(self):
```

```
        print("Estoy dando el radio")
```

```
        return self._radio
```

```
    @radio.deleter
```

```
    def radio(self):
```

```
        print("Borrando el radio...")
```

```
        del self._radio
```

```
# Ejemplo de uso
```

```
c1 = Circulo(3)
```

```
print(c1.radio) # Imprime: Estoy dando el radio y luego el valor 3
```

```
del c1.radio    # Elimina el atributo _radio
```

```
print(c1.radio) # Lanza AttributeError porque _radio ya no existe
```

4. Herencia

4.1 Herencia simple

Una clase puede heredar atributos y métodos de otra.

Ejemplo:

```
class Animal:
    def hablar(self):
        print("El animal hace un sonido ...")

class Perro(Animal):
    def ladrar(self):
        # Llamamos al método heredado de la clase padre
        super().hablar()
        # Agregamos el comportamiento específico del perro
        print("El perro ladra.")

# Crear un objeto de la clase hija
p = Perro()
p.ladrar()
```

4.2 Uso de super()

El uso principal de la función `super()` en Python es **llamar a un método de la clase padre (o superclase)** desde un método de la clase hija (o subclase). Esto es especialmente útil en dos escenarios clave:

1. **Invocar el constructor del padre (`__init__`):** Es muy común que una clase hija necesite hacer lo que su padre hace al inicializarse, además de sus propias tareas de inicialización. Esto asegura que todos los atributos de la clase base se configuren correctamente.
2. **Llamar a un método sobrescrito del padre:** Si una clase hija redefine un método que también existe en su padre (polimorfismo), `super()` permite llamar a la versión del padre

de ese método. Esto es útil si quieres añadir funcionalidad a un método heredado sin reescribirlo por completo, o simplemente referenciar la implementación original.

Ejemplo:

```
class Animal:
```

```
    def __init__(self, nombre):
        self.nombre = nombre
        print(f"Se inicializa un Animal llamado {self.nombre}")
```

```
    def hablar(self):
        """Método genérico para hacer un sonido."""
        print(f"{self.nombre} hace un sonido genérico.")
```

```
class Perro(Animal):
```

```
    def __init__(self, nombre, raza):
        # 1. Usando super() para llamar al constructor de la clase padre (Animal)
        # Esto asegura que 'self.nombre' se inicialice correctamente en la clase base.
        super().__init__(nombre)
        self.raza = raza
        print(f"Se inicializa un Perro de raza {self.raza}")
```

```
    def hablar(self):
        """Sobrescribe el método hablar para un Perro."""
        print(f"{self.nombre} (raza {self.raza}) ladra: ¡Guau!")
```

```
    def interactuar(self):
        """
        Método que demuestra llamar tanto a su propia versión de 'hablar()'
        como a la versión de la clase padre.
        """
        print(f"\n--- {self.nombre} va a interactuar ---")
```

```
# Llama al método 'hablar' de la clase Perro (su propia implementación sobrescrita)
self.hablar()
```

```
# 2. Usando super() para llamar a un método sobrescrito del padre
```

```
# Accedemos a la implementación de 'hablar()' de la clase Animal,
# incluso después de que Perro haya definido su propia versión.
```

```
print("(Pero si este perro fuera un Animal, haría lo siguiente:)")
super().hablar()
```

```
print(f"--- Fin de la interacción de {self.nombre} ---")
```

```
# --- Demostración ---
```

```
print("Creando una instancia de Perro:")
```

```
mi_perro = Perro("Fido", "Golden Retriever")
```

```
print("\nUsando el método hablar del Perro:")
```

```
mi_perro.hablar() # Llama a Perro.hablar()
```

```
print("\nDemostrando la interacción, que usa super().hablar():")
```

```
mi_perro.interactuar()
```

```
# También podemos crear una instancia de la clase padre para comparar
```

```
print("\nCreando una instancia de Animal:")
```

```
mi_animal = Animal("Leon")
```

```
mi_animal.hablar() # Llama a Animal.hablar()
```

4.3 Herencia múltiple

Una clase puede heredar de varias clases base.

```
class A:
```

```
    def metodo(self):  
        print("A")
```

```
class B:
```

```
    def metodo(self):  
        print("B")
```

```
class C(A, B):
```

```
    pass
```

```
c = C()
```

```
c.metodo() # Salida: A
```

```
# El motivo es que llama al método de A por orden de herencia (debido al MRO: C -> A -> B)
```

```
# Ver explicación del concepto MRO al final del documento.
```

```
# Para llamar el método de B, lo hacemos directamente a través de la clase B,
```

```
# pasándole la instancia 'c' como si fuera 'self'.
```

```
B.metodo(c) # Salida: B
```

4.4. Funciones `issubclass()` y `isinstance()`

La función `issubclass(SubClase, ClaseSup)` se utiliza para comprobar si una clase (SubClase) es hija de otra superior (ClaseSup), devolviendo True o False según sea el caso.

```
issubclass(Movil, Telefono)
```

La función booleana `isinstance(Objeto, Clase)` se utiliza para comprobar si un objeto pertenece a una clase o clase superior.

```
isinstance(mimovil, Movil)
```

4.5 Delegación

Se llama delegación a la situación en la que una clase contiene (como atributos) una o más instancias de otra clase, a las que delegará parte de sus funcionalidades. En otras palabras, en lugar de heredar de otra clase, **usa sus métodos y atributos**.

Por ejemplo:

- Un Círculo **tiene un** Punto como centro.
- Por eso, la clase Círculo no hereda de Punto, sino que **contiene un objeto Punto**.

Ejemplo:

class Punto:

```
def __init__(self, x, y):  
    self.x = x  
    self.y = y
```

class Circulo:

```
def __init__(self, centro, radio):  
    self.centro = centro # objeto de tipo Punto  
    self.radio = radio
```

Crear un objeto Punto y un objeto Circulo

c1 = Circulo(Punto(2, 3), 5)

Imprimir información del círculo directamente usando los atributos

print("Centro:", c1.centro.x, ":", c1.centro.y, "- Radio:", c1.radio)

5. Polimorfismo

El mismo método puede comportarse de forma distinta según el objeto.

--- 1. Clase Base (Superclase) ---

```
class Vehiculo:
```

```
    """ Define un comportamiento genérico para cualquier vehículo. Esta es la interfaz que otras
    clases de vehículos seguirán. """
```

```
    def arrancar(self):
```

```
        """ Método genérico de arranque para un Vehículo. Será sobrescrito por las clases hijas
        para dar un comportamiento específico. """
```

```
        print("Vehículo en marcha")
```

--- 2. Clases Derivadas (Subclases) ---

```
class Coche(Vehiculo):
```

```
    """ Clase Coche: Hereda de Vehiculo. Un Coche "es un" Vehiculo, pero tiene su propia forma
    particular de arrancar. """
```

```
    def arrancar(self):
```

```
        """ Sobrescribe el método 'arrancar()' de la clase base Vehiculo. Ahora, cuando un objeto
        Coche llama a 'arrancar()', ejecutará esta versión. """
```

```
        print("Coche en marcha")
```

```
class Moto(Vehiculo):
```

```
    """Clase Moto: También hereda de Vehiculo. Una Moto "es un" Vehiculo, y también define su
    propia forma de arrancar. """
```

```
    def arrancar(self):
```

```
        """ Sobrescribe el método 'arrancar()' para Moto. Cuando un objeto Moto llama a
        'arrancar()', ejecutará esta versión. """
```

```
        print("Moto en marcha")
```

--- 3. Función Polimórfica ---

```

def iniciar(v):
    """ Esta función demuestra el polimorfismo. Puede aceptar cualquier objeto 'v' (vehículo)
    que tenga un método 'arrancar()'. No necesita saber si 'v' es un Coche, una Moto, etc. """
    v.arrancar()

    """ Aquí ocurre el polimorfismo: el mismo método 'arrancar()' se comporta de forma
    diferente según el tipo real de 'v'. """

# --- 4. Demostración del Polimorfismo ---

print("--- Iniciando diferentes vehículos ---")

# Creamos una instancia de Coche y la pasamos a la función 'iniciar'
# 'v' dentro de 'iniciar' será un objeto Coche.
iniciar(Coche()) # Se imprimirá "Coche en marcha" porque el 'arrancar()' de Coche es llamado.

print("---")

# Creamos una instancia de Moto y la pasamos a la función 'iniciar'
# 'v' dentro de 'iniciar' será un objeto Moto.
iniciar(Moto()) # Se imprimirá "Moto en marcha" porque el 'arrancar()' de Moto es llamado.

print("---")

# Incluso podemos pasar una instancia de la clase base Vehiculo
# 'v' dentro de 'iniciar' será un objeto Vehiculo.
iniciar(Vehiculo()) # Se imprimirá "Vehículo en marcha" porque el 'arrancar()' de Vehiculo es
llamado.

```

6. Abstracción

Las clases abstractas definen una interfaz que deben implementar las subclases. Su propósito es **definir una interfaz común** para un grupo de clases, ocultando los detalles de implementación específicos de cada una. Es como un conjunto de reglas que todas las clases que hereden de una clase abstracta deben seguir.

Ejemplo:

```
# Importamos ABC y abstractmethod del módulo 'abc'
```

```
# Estas son las herramientas que Python nos da para trabajar con clases abstractas.
```

```
from abc import ABC, abstractmethod
```

```
class Figura(ABC):
```

```
    """ Esta es nuestra Clase Base Abstracta (ABC). Heredar de 'ABC' le dice a Python que esta
    clase es abstracta y que puede contener métodos abstractos. NO se pueden crear instancias
    directamente de una clase abstracta si tiene métodos abstractos sin implementar. """
```

```
    @abstractmethod
```

```
    def area(self):
```

```
        """
```

```
        Este es un método abstracto. El decorador '@abstractmethod' obliga a CUALQUIER
        subclase concreta de 'Figura' a
```

```
        implementar su propia versión de este método 'area()'. Aquí solo definimos que el método
        debe existir, pero no cómo funciona.
```

```
        'pass' indica que no hay implementación en la clase abstracta.
```

```
        """
```

```
        pass
```

```
# --- 2. Clases Concretas (Subclases que implementan la interfaz abstracta) ---
```

```
class Cuadrado(Figura):
```

```
    """ Esta es una clase concreta que hereda de Figura. Como es concreta, está OBLIGADA a
    implementar todos los métodos abstractos de su clase padre abstracta (en este caso, 'area()').
    """
```

```
def __init__(self, lado):
```

```
    self.lado = lado
```

```
def area(self):
```

```
    """ Implementación CONCRETA del método 'area()' para la clase Cuadrado. Aquí es donde  
    se proporciona la lógica real para calcular el área de un cuadrado. """
```

```
    return self.lado ** 2
```

```
# class Circulo(Figura):
```

```
# """ # Si tuviéramos otra figura como Círculo, también tendría que implementar 'area()'.
```

```
# Esto demuestra cómo la clase abstracta 'Figura' fuerza una interfaz común.
```

```
# """
```

```
# def __init__(self, radio):
```

```
#     self.radio = radio
```

```
#
```

```
# def area(self):
```

```
#     return 3.14159 * (self.radio ** 2)
```

```
# --- 3. Uso de las Clases ---
```

```
c = Cuadrado(5)
```

```
print(c.area()) # 25
```

7. Gestión de recursos y destrucción de objetos

7.1 Método `__del__`

Se ejecuta cuando un objeto es destruido.

```
class Prueba:
```

```
    def __del__(self):  
        print("Objeto destruido")
```

```
p = Prueba()
```

```
del p # Imprime "Objeto destruido"
```

El método `__del__` permite realizar acciones de limpieza antes de que un objeto sea eliminado, como liberar memoria o cerrar conexiones.

No obstante, su uso debe ser limitado. En Python, la destrucción de objetos no siempre ocurre en el momento exacto en que se elimina una variable, sino se detecta que el objeto ya no tiene referencias. Por este motivo, no es recomendable depender de `__del__` para liberar recursos críticos, como archivos o conexiones abiertas. En estos casos, resulta más adecuado emplear **gestores de contexto** mediante la instrucción `with`, que garantizan una liberación inmediata y controlada de los recursos.

7.2 Gestores de contexto y método `with`

Un **gestor de contexto** es un objeto que controla la inicialización y liberación de recursos.

Se utiliza junto con la instrucción `with`, que garantiza que los recursos se liberen correctamente, incluso si ocurre un error durante su uso.

Los gestores de contexto se definen mediante los métodos especiales `__enter__` y `__exit__`.

Ejemplo:

```
class Recurso:
```

```
def __enter__(self):  
    print("Recurso inicializado")  
    return self  
  
def __exit__(self, tipo, valor, traza):  
    print("Recurso liberado")
```

```
with Recurso():  
    print("Usando el recurso")
```

Salida:

Recurso inicializado

Usando el recurso

Recurso liberado

El bloque with ejecuta automáticamente el método `__enter__` al comenzar y el método `__exit__` al finalizar, asegurando la correcta gestión de los recursos.

Los parámetros `tipo`, `valor` y `traza` son proporcionados por el sistema si ocurre una excepción dentro del bloque:

- `tipo` indica la clase de la excepción,
- `valor` contiene el mensaje o instancia del error,
- `traza` proporciona información sobre dónde ocurrió el fallo.

Si el método `__exit__` devuelve `True`, la excepción se considera gestionada y no se propaga; si devuelve `False` o no devuelve nada, la excepción se lanza normalmente.

8. Métodos especiales en Python

Los **métodos especiales** (también conocidos como "métodos mágicos" o "dunder methods" por el doble guion bajo que los precede y los sigue) en Python son funciones predefinidas que nos permiten personalizar el comportamiento de nuestras clases y objetos.

Estos métodos se invocan automáticamente en ciertas situaciones, como al imprimir un objeto, al compararlo o al realizar operaciones aritméticas con él.

8.1 Representación de objetos: `__str__` y `__repr__`

Estos dos métodos son fundamentales para controlar cómo se muestran los objetos de una clase. Son especialmente útiles para la depuración y para proporcionar información clara al usuario.

- `__str__(self)`: Este método debe devolver una **representación informal y legible para el usuario** del objeto. Es lo que se muestra cuando utilizas la función `print()` sobre una instancia de una clase. Es como una descripción amigable que explica el estado actual del objeto de manera concisa.
- `__repr__(self)`: Por otro lado, `__repr__` debe devolver una **representación formal e inambigua** del objeto. Su propósito principal es ser útil para los desarrolladores. Se utiliza cuando se evalúa el objeto directamente en la consola de Python o cuando se llama explícitamente a la función `repr()`.

A continuación, se muestra un ejemplo aplicado a una hipotética clase `Circulo`:

```
class Circulo:
```

```
    def __init__(self, radio):
```

```
        self.radio = radio
```

```
    def __str__(self):
```

```
        # Devuelve una cadena fácil de entender para el usuario
```

```
        clase = type(self).__name__
```

```
        return f"{clase} de radio {self.radio}"
```

```
def __repr__(self):  
    # Devuelve una representación que podría recrear el objeto  
    clase = type(self).__name__  
    return f"{clase}({self.radio})"
```

Ejemplo de uso:

```
c1 = Circulo(3)
```

Uso de __str__ (invocado por print())

```
print(c1)    # Salida: Circulo de radio 3
```

Uso de __repr__ (invocado por repr() o al evaluar en consola)

```
print(repr(c1))    # Salida: 'Circulo(3)'
```

Demostración de reconstrucción del objeto usando eval() y repr()

```
c2 = eval(repr(c1)) # c2 es ahora un nuevo objeto Circulo(3)
```

```
print(c2)    # Salida: Circulo de radio 3
```

```
print(c1 is c2) # Salida: False (son objetos diferentes, pero con el mismo valor)
```

En este ejemplo, __str__ nos da una descripción legible de "Circulo de radio 3", ideal para mostrar al usuario. Mientras tanto, __repr__ nos da "Circulo(3)", una representación que permite a un desarrollador ver rápidamente cómo se podría haber creado ese círculo y, en este caso, incluso recrearlo con eval().

8.2 Comparación de objetos: `__eq__` y operadores relacionales

Por defecto, cuando se comparan dos objetos en Python con `==`, se compara **su identidad**, es decir, si son **el mismo objeto en memoria**, no sus atributos.

Si se desea comparar objetos según el **valor de sus atributos**, es necesario redefinir el método especial `__eq__`.

- `__eq__(self, otro)`: se invoca al usar `==` entre dos objetos. Debe devolver `True` si los objetos se consideran iguales según los atributos relevantes, y `False` en caso contrario.

```
class Circulo:
```

```
    def __init__(self, radio):
```

```
        self.radio = radio
```

```
    def __eq__(self, otro):
```

```
        # Comprueba si el otro objeto también es un Circulo y si sus radios son iguales
```

```
        if isinstance(otro, Circulo):
```

```
            return self.radio == otro.radio
```

```
        return NotImplemented # Indica que la comparación no está implementada para este tipo
```

```
# Ejemplo de uso:
```

```
c1 = Circulo(5)
```

```
c2 = Circulo(5)
```

```
c3 = Circulo(7)
```

```
print(c1 == c2) # Salida: True (sus radios son iguales)
```

```
print(c1 == c3) # Salida: False (sus radios son diferentes)
```

```
print(c1 == "un circulo") # Salida: False (se maneja el tipo con isinstance)
```

Operadores Relacionales Adicionales:

Si además de la igualdad, deseas que los objetos puedan ser comparados utilizando otros operadores relacionales como `<`, `<=`, `>`, o `>=`, se debe redefinir los métodos correspondientes:

- `__lt__(self, otro)` para el operador `<` (menor que)

- `__le__(self, otro)` para el operador `<=` (menor o igual que)
- `__gt__(self, otro)` para el operador `>` (mayor que)
- `__ge__(self, otro)` para el operador `>=` (mayor o igual que)

Al implementar estos métodos, se proporciona un comportamiento coherente para todas las comparaciones de objetos. Por ejemplo, en la clase `Circulo`, se podría definir que un círculo es "menor" que otro si su radio es menor.

```
def __lt__(self, otro):  
    if isinstance(otro, Circulo):  
        return self.radio < otro.radio  
    return NotImplemented
```

Con esto, `c1 < c2` funciona según el valor de radio.

8.3 Operaciones con objetos: `__add__` y `__sub__`

En Python, es posible redefinir los operadores aritméticos (+, -, etc.) para objetos personalizados mediante **métodos especiales**. Esto permite definir cómo se comporta un objeto al sumarse o restarse con otro objeto de la misma clase.

- `__add__(self, otro)` → se invoca cuando se usa + entre objetos.
- `__sub__(self, otro)` → se invoca cuando se usa - entre objetos.

Redefinir estos métodos permite que los objetos interactúen de manera coherente según su lógica interna.

Ejemplo:

class Circulo:

```
def __init__(self, radio):  
    self.radio = radio
```

```
# Redefinir el operador +
```

```
def __add__(self, otro):  
    if isinstance(otro, Circulo):  
        # Devuelve un nuevo Circulo con la suma de radios  
        return Circulo(self.radio + otro.radio)  
    return NotImplemented
```

```
# Redefinir el operador -
```

```
def __sub__(self, otro):  
    if isinstance(otro, Circulo):  
        nuevo_radio = self.radio - otro.radio  
        if nuevo_radio >= 0:  
            return Circulo(nuevo_radio)  
        else:  
            raise ValueError("Resultado negativo no permitido")  
    return NotImplemented
```

```
# Uso de los operadores redefinidos
```

```
c1 = Circulo(5)
```

```
c2 = Circulo(3)
```

```
c3 = c1 + c2
```

```
print(c3.radio) # 8
```

```
c4 = c3 - c2
```

```
print(c4.radio) # 5
```

8.4 Otros métodos especiales

Python es increíblemente flexible y permite la personalización del comportamiento de las clases a un nivel muy profundo a través de una amplia gama de métodos especiales.

Además de los vistos, existen métodos para:

- **Colecciones:** `__len__` (para la función `len()`), `__getitem__` (para acceder a elementos por índice o clave como en listas y diccionarios), `__setitem__`, `__delitem__`, `__contains__` (para el operador `in`).
- **Gestores de contexto:** `__enter__` y `__exit__` (para usar la sentencia `with`).
- **Callables:** `__call__` (para hacer que un objeto pueda ser invocado como una función).
- **Conversión de tipos:** `__int__`, `__float__`, `__bool__`.
- Etc.

La capacidad de redefinir estos métodos permite crear objetos que se comportan de manera similar a los tipos de datos nativos de Python, lo que hace que el código sea más fácil de entender.

Qué es el mangling

El **mangling de nombres** (o "name mangling" en inglés) es un mecanismo que utiliza Python para hacer que los atributos y métodos "privados" (aquellos que comienzan con **dos guiones bajos** `__`) sean más difíciles de acceder directamente desde fuera de la clase.

No es una característica de seguridad estricta, sino más bien una forma de **evitar colisiones de nombres** y de indicar fuertemente que un atributo o método está destinado solo para **uso interno** de la clase.

¿Cómo funciona?

Cuando defines un atributo o método con doble guion bajo, Python reescribe internamente su nombre añadiendo un prefijo que incluye un guion bajo y el nombre de la clase.

Por ejemplo, si tienes una clase `MiClase` y un atributo `__secreto`:

Python

```
class MiClase:
```

```
    def __init__(self):
```

```
        self.__secreto = "Esto es muy privado"
```

```
objeto = MiClase()
```

```
# Intentar acceder directamente causará un error
```

```
# print(objeto.__secreto)
```

Python transforma `__secreto` en algo como `_MiClase__secreto`. Así, para acceder a él desde fuera, tendrías que usar el nombre "mangled": `objeto._MiClase__secreto`.

¿Por qué se hace esto?

1. **Evitar Colisiones de Nombres en Herencia:** Si tienes una clase base y una subclase, ambas con un atributo `__privado`, el mangling asegura que no se sobrescriban accidentalmente. Cada atributo "privado" se "mangea" con el nombre de su propia clase, garantizando que sean únicos.
2. **Indicador de Uso Interno:** Sirve como una señal clara para otros desarrolladores de que ese atributo o método es un detalle de implementación interna de la clase y no debe ser

manipulado directamente desde el exterior. Si necesitas acceder a él, es probable que estés rompiendo el diseño de la clase.

Observación: aunque el mangling dificulta el acceso, no lo impide por completo. Si un programador conoce el nombre "mangled", puede acceder al atributo.

¿Qué es el MRO (Method Resolution Order)?

El **Method Resolution Order (MRO)** es un concepto fundamental en la programación orientada a objetos de Python, especialmente relevante en escenarios de **herencia múltiple**. Es el mecanismo que Python utiliza para determinar el orden en que se buscan los métodos y atributos en la jerarquía de clases cuando una instancia llama a un método. En otras palabras, **establece el camino** que Python sigue para encontrar la implementación correcta de un método.

Cuando un objeto de una clase llama a un método, Python no busca la implementación de forma aleatoria. En su lugar, sigue un algoritmo específico, conocido como C3 linearization, para construir el MRO de esa clase. Este orden es una secuencia lineal de clases que define el orden de búsqueda.

Considerando la definición de clase que se presentó previamente:

```
class A:
```

```
    def metodo(self):  
        print("A")
```

```
class B:
```

```
    def metodo(self):  
        print("B")
```

```
class C(A, B):
```

```
    pass
```

Para una clase como C, que hereda de A y B en ese orden (class C(A, B):), el MRO de C se establece como (C, A, B, object). Esto significa que, al intentar resolver la llamada a un método (por ejemplo, c.metodo()):

1. Python **primero buscará la implementación del método en la propia clase C**.
2. Si el método no se encuentra en C (como ocurre en el ejemplo, donde C tiene pass y no define metodo), **pasará a la siguiente clase en el MRO, que es A**.
3. Al encontrar metodo() en la clase A, **ejecutará esa implementación**.
4. Debido a que ya encontró una implementación en A, **nunca llegará a buscar el método en B** para esa llamada particular.